

# PA01

Name: Ayush Regmi

Section: 01

## Question 1

### 1. Steps Cost

- Formula:  $h(n) = k$  ( number of mismatched letters between the current state and goal state).
- Lower bound: It lower bounds the minimum number of actions required to reach the goal.
- $h(n) = k$  can never overestimate because for k number of mismatched letters, it will at least takes k steps to reach the goal.

### 2. Scrabble Cost

- Formula:  $h(n) = \text{sum}(\text{SCRABBLE\_VALUES}[\text{goal}[i]] \text{ for every index } i \text{ where } \text{state}[i] \neq \text{goal}[i])$
- Lower bound: It lower bounds the total Scrabble point cost of inserting the letters to form the goal word
- The cost of an action is determined by the value of the newly inserted letter. To reach the goal, every incorrect position must be replaced by the correct target letter. This calculates the cost of inserting those letters. Since there are no intermediate steps added in the formula, this function strictly bounds the cost by just summing the necessary steps.

### 3. Frequency Cost

- Formula:  $h(n) = k$  ( number of mismatched letters between the current state and goal state).
- Lower bound: It lower bounds the cumulative minimum step cost of the remaining path.
- The cost of producing a new word  $w$  is  $1 + \text{rarity}(w)$ . Because the dictionary's rarity values are nonnegative, the absolute minimum cost of any single action is  $1 + 0 = 1$ . And fixing  $k$  mismatched letters require letters requires at leas  $k$  steps, and the cheapest possible step costs 1, total remaining cost must be  $\geq k$ .

## Question 2

| Word pair   | Cost measure | UCS cost | UCS nodes expanded | A* cost | A* nodes expanded | Reduction (%) |
|-------------|--------------|----------|--------------------|---------|-------------------|---------------|
| WHY → ASK   | Steps        | 5        | 773                | 5       | 16                | 97.93         |
| WHY → ASK   | Scrabble     | 9        | 765                | 9       | 13                | 98.30         |
| WHY → ASK   | Frequency    | 38.855   | 808                | 38.855  | 755               | 6.60          |
| BEAR → DUCK | Steps        | 4        | 691                | 4       | 6                 | 99.13         |
| BEAR → DUCK | Scrabble     | 11       | 509                | 11      | 54                | 89.39         |
| BEAR → DUCK | Frequency    | 38.240   | 1528               | 38.240  | 1367              | 10.54         |

1. Yes, UCS and A\* always returned the exact same optimal cost across all test cases.
2. For WHY → ASK, the Scrabble measure produced the largest reduction (98.30%). For BEAR → DUCK, the Steps measure produced the largest reduction (99.13%).
3. The Frequency measure was least informative for both pairs, with 6.60% and 10.54% reduction of node expansion.
4. The Steps and Scrabble heuristics closely approximate the true remaining cost by calculating the exact minimum moves, which heavily reduces the size of the search tree. But, the Frequency heuristic only counts mismatched letters while ignoring the actual rarity penalties. It underestimates the true path cost and A\* gets very little information and explores almost as many nodes as UCS.

## Question 3

### 1. Steps Cost

- Symmetric: Yes
- If a valid path from  $W$  to  $W'$  takes  $N$  steps, the exact reverse path from  $W'$  to  $W$  takes exactly  $N$  steps.

### 2. Scrabble Cost

- Symmetric: No
- Transitioning from `dog` to `cog` requires inserting `c`, which costs 3
- Transitioning from `cog` to `dog` requires inserting `d`, which costs 2.

### 3. Frequency Cost

- Symmetric: No
- Transitioning from `dog` to `cog` costs  $1 + \text{rarity}(\text{"cog"})$
- Transitioning from `cog` to `dog` costs  $1 + \text{rarity}(\text{"dog"})$

Because they have different rarity value, the total path costs will not be equal.

## Question 4

### 1. Model Used: ChatGPT

### 2. Prompt (from the assignment file):

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and the frequency objective, entering word  $w$  costs  $1 + \text{rarity}(w)$ , where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic  $h_0(n)$  is the number of positions in which state  $n$  differs from the goal. Propose a rarity-aware heuristic intended to be more informative than  $h_0$  while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as  $h_0$  for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

### 3. Proposed Heuristic:

ChatGPT proposed finding the absolute smallest rarity values in the entire dictionary ( $\rho_1, \rho_2, \dots$ ) and adding the sum of the  $k$  smallest rarities to the baseline step count, where  $k$  is the number of mismatched letters.

### Proposed Formula by ChatGPT:

Let  $d(n)$  be the number of mismatched letters (Hamming distance) between the current state and the goal. Let  $\rho_1 \leq \rho_2 \leq \dots \leq \rho \mid D$  be the sorted rarity values of every word in the dictionary.

$$h_R(n) = d(n) + \sum_{i=1}^{d(n)} \rho_i$$

(If  $d(n) = 0$ , the heuristic returns 0).